

Working with variables

We can define variables to store information, which can be accessed within the shell script. There are certain rules we need to follow while defining variables:

1. The variable name can contain any combination of letters from A-Z or a-z, digits from 0-9 or an underscore (_) character.
 2. The variable name should start with a letter or an underscore character.
- A few examples of valid variables are given below:

```
NAME
name
_name_
name123
```

By convention, shell variables are defined in upper case. Given below is a simple script, which shows the usage of a variable within it:

```
$ cat greet.sh
NAME="Narendra"
echo "Welcome $NAME"
```

In the above example, we have defined the 'NAME' variable, which holds the value 'Narendra'. Note that we have to use a dollar (\$) symbol with a variable name to access it. The above script generates the following output when it is executed:

```
$ bash greet.sh
Welcome Narendra
```

In this output, the 'echo' command has substituted the value of the variable 'NAME'. Please note that variable names are case sensitive. So all of the variables given below are treated as different variables:

1. NAME
2. Name
3. NaMe, and so on...

Conditional expressions

Like other programming languages,

shell supports conditional expressions like - *if*, *if-else* and *case*. Let us understand this with simple examples.

- a) *if* expression: In case we want to verify whether a file exists or not, we can use the *if* expression, as follows:

```
$ touch file.txt #this creates a file

$ cat file_existance.sh
if [ -e file.txt ]; then
    echo "file.txt file exists"
fi
```

The script given above generates the following output when it is executed:

```
$ bash file_existance.sh
file.txt file exists
```

- b) *if-else* expression: Let us modify the above script to generate output when the file does not exist:

```
$ rm file.txt

$ cat file_existance.sh
if [ -e file.txt ]; then
    echo "file.txt file exists"
else
    echo "file.txt file does not exists"
fi

$ bash file_existance.sh
file.txt file does not exists
```

- c) *case* statement: Shell provides a switch statement-like functionality using the *case* statement. Let us understand this with the example given below:

```
$ cat case_statement.sh
read -p "Enter the day: " DAY

case $DAY in
    Mon) echo "Today is Monday"
    ;;

    Tue) echo "Today is Tuesday"
    ;;
```

```
Wed) echo "Today is Wednesday"
;;

*) echo "Unknown day"
;;
esac
```

The script given above generates the following output when it is executed:

```
$ bash case_statement.sh
Enter the day: Mon
Today is Monday

$ bash case_statement.sh
Enter the day: Tue
Today is Tuesday

$ bash case_statement.sh
Enter the day: Invalid
Unknown day
```

In *case* statement, asterisk (*) denotes a default case.

Looping constructs

Like other programming languages, shell supports looping constructs. In this section, we will discuss four loops supported by shell.

- a) *for* loop: This operates on lists of items and repeats a set of commands for every item in a list. For example, the script given below prints characters from A to Z.

```
$ cat loop.sh
for char in {A..Z}
do
    echo -n "$char "
done
```

The above script generates the following output when it is executed:

```
$ bash loop.sh

A B C D E F G H I J K L M N O P Q R S T
U V W X Y Z
```

- b) *while* loop: This executes a set of commands until the provided